

AI 学习助手知识库测试资料

一份用于测试 ChatPDF / RAG / Agent 的学习文档

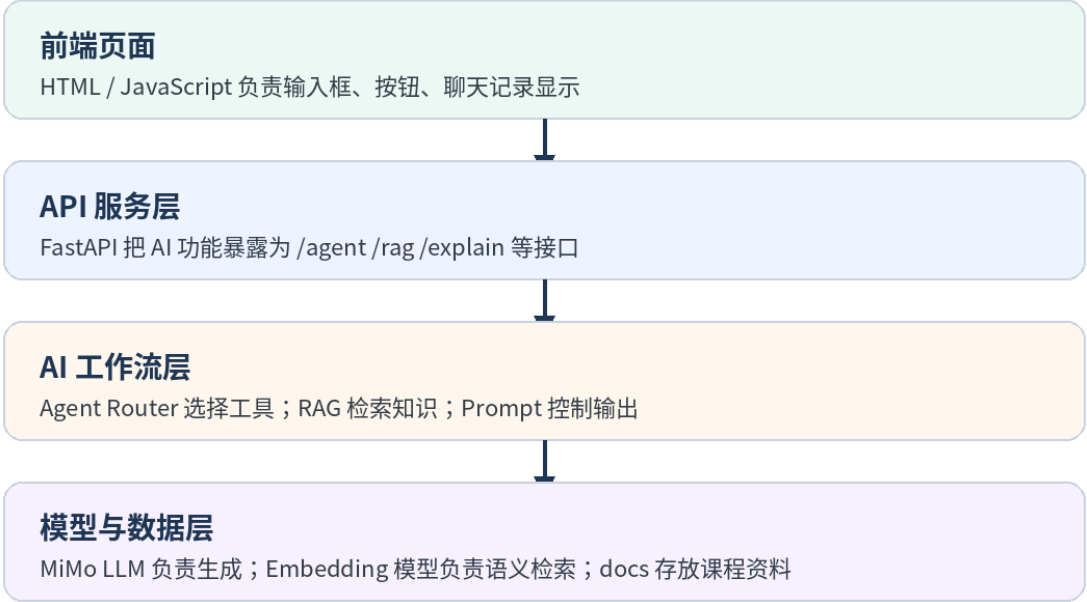
这份 PDF 专门用于你的 AI 学习助手项目。它包含清晰的章节、表格和示意图，适合用来测试：PDF 读取、chunk 切分、embedding 检索、RAG 回答、Agent 路由和 FastAPI 服务化。

阶段	你现在要掌握的能力	对应项目功能
1. Prompt 与聊天	写清楚指令，理解 system/user/assistant 消息	explain、chat
2. RAG 知识库	读取 txt/pdf/md，检索相关内容再回答	rag_answer
3. Agent 路由	让模型判断任务并调用工具	agent_router
4. 服务化	把 AI 能力变成 API，供网页调用	FastAPI /agent /rag
5. 产品化	上传 PDF、显示来源、聊天界面	HTML + upload + ChatPDF

1. 你现在的项目是什么

你正在做的不是一个简单的 API 调用脚本，而是一个小型 AI 应用系统。它由前端页面、FastAPI 后端、AI 核心函数、RAG 检索模块、Agent 路由模块和文档知识库组成。

AI 学习助手系统分层图



理解这个分层很重要：前端负责展示和收集输入；FastAPI 负责接收请求并返回 JSON；ai_core.py 负责真正调用模型、检索文档和执行工具。

你应该能解释的三个问题

- 为什么前端不能直接调用 Python 函数？
- 为什么 FastAPI 要 return JSON，而不是 print？
- 为什么 RAG 函数必须返回字符串，才能被 API 调用？

2. RAG 为什么会错

RAG 的关键不是“让模型多聪明”，而是“先找到正确资料”。如果检索到错误 chunk，LLM 就会根据错误资料回答。你之前看到回答混乱，很多时候不是模型不会，而是 retrieval 找到的内容太碎、太脏或不相关。

RAG：先检索，再生成

关键：回答质量很大程度取决于检索质量，而不只是模型大小。



常见错误来源

问题	表现	解决方向
chunk 太小	只检索到标题或半句话	增大 chunk_size，加入 overlap
文档太脏	检索到图片路径、目录、链接噪音	清洗 Markdown 和无关内容
Top-K 不合适	只取一个片段导致信息不全	取 Top 3 或 Top 5
知识库没有内容	回答“知识库中没有相关内容”	导入更高质量课程资料
Prompt 太宽松	模型自由发挥	要求严格根据知识回答

3. Retrieval 为什么重要

Retrieval 是 RAG 的“找资料”环节。它决定了 LLM 看到什么上下文。你可以把 RAG 想成开卷考试：模型负责答题，retrieval 负责翻书。如果翻错页，再强的模型也会答偏。

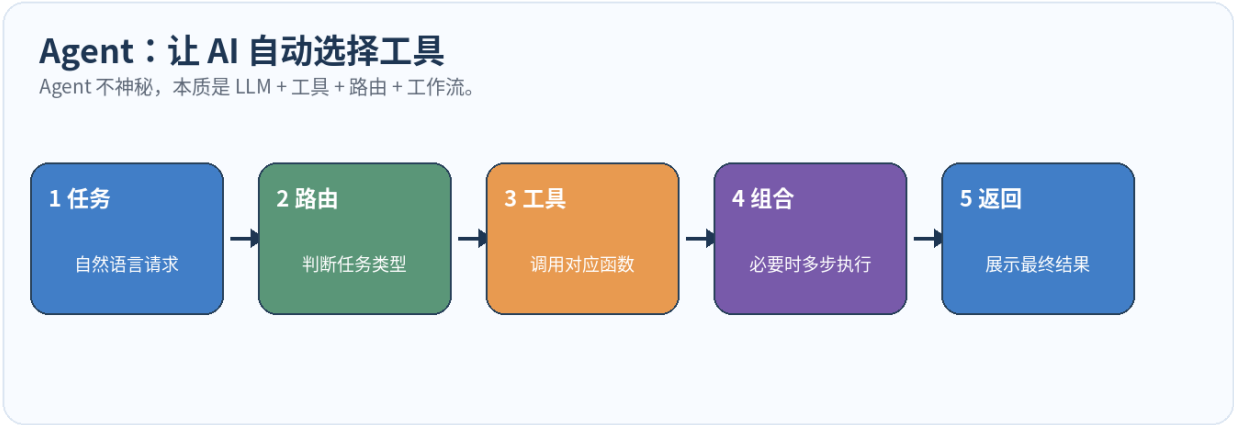
在你的项目中，retrieval 主要由三步构成：第一，把文档切成 chunks；第二，用 SentenceTransformer 生成 embedding；第三，用 cosine_similarity 找 Top-K 最相似片段。

概念	在你项目里的对应代码	通俗理解
Embedding	embedding_model.encode(...)	把文字变成“语义坐标”
Similarity	cosine_similarity(...)	计算问题和文档片段有多像
Top-K	argsort()[-top_k:]	取最相关的几个片段
Context	relevant_text	提供给 LLM 的参考资料
Answer	llm.invoke(prompt)	根据参考资料组织回答

调试 RAG 时，先看“检索到的知识”，再看 AI 回答。如果检索结果不对，优先改文档、chunk 或 embedding，而不是马上怪模型。

4. Agent 本质是什么

Agent 不是神秘的“有意识 AI”。在你的项目里，Agent 的本质很清楚：先判断用户意图，再选择对应工具函数。比如用户说“帮我解释 Embedding”，Agent 选择 explain；用户问课程资料，Agent 选择 rag_answer。



你项目里的工具映射

用户请求类型	Agent 选择	执行函数
普通聊天	chat	多轮对话
解释概念	explain	用通俗中文解释
总结内容	summarize	压缩重点
生成练习题	generate_questions	出题并给答案
基于文档回答	rag_answer	检索知识库再回答

5. FastAPI 在你的系统里负责什么

FastAPI 不负责“变聪明”，它负责把 AI 能力变成可以通过网络调用的接口。以前你只能在终端运行 app.py；现在网页、React、手机 App 或其他程序都可以通过 HTTP 请求调用你的 AI。

典型请求流程

- 前端 fetch("http://127.0.0.1:8000/agent") 发出请求。
- FastAPI 根据 @app.post("/agent") 找到对应函数。
- Pydantic 把 JSON 中的 text 解析成 request.text。
- server.py 调用 ai_core.py 中的 agent_router。
- ai_core.py 调用 LLM、RAG 或工具函数。
- FastAPI 把结果包装成 JSON 返回给前端。

所以，FastAPI 是“AI 服务入口”。它让你的 AI 不只是本地脚本，而是一个真正可以被网页调用的服务。

6. 你现在下一步该优化什么

优先级	升级方向	为什么做
高	上传 PDF 后自动刷新向量缓存	否则新文件上传后可能不会立刻被检索
高	显示来源文件名和 chunk 编号	方便判断答案来自哪里
中	清洗 Markdown 噪音	减少图片路径和链接干扰
中	返回 sources 给前端	让网页显示引用来源
进阶	使用 FAISS 或 ChromaDB	避免每次全量相似度计算